Cache Timing Side-Channel Analysis of T-Table AES on RISC-V

Gianmarco Fortunelli

Supervision: Prof. R. Pacalet

Sophia Antipolis, January 2026

Abstract

Cache-based side-channel attacks on AES are well understood in theory and have been demonstrated on a wide range of mainstream architectures. However, the extent to which these attacks remain effective under realistic execution conditions and on less-studied platforms is still not fully characterized.

In this work, we present an experimental replication and systematic evaluation of access-driven cache attacks on a modern RISC-V system-on-chip. Focusing on a Prime+Probe attacker targeting the private L1 data cache, we analyze the observability of AES T-table memory accesses at cache-set granularity under controlled but realistic operating system conditions. We explicitly separate leakage detection, statistical classification, and key inference, and study how noise, cache-set heterogeneity, and measurement aggregation affect each stage.

Our results show that cache access patterns remain observable at the per-set level and allow reliable recovery of the most significant bits of the AES key bytes, while structural limitations of the cache geometry and T-table layout prevent full key recovery from single-round leakage alone. We further demonstrate that leakage quality varies significantly across cache sets, highlighting the importance of per-set analysis when assessing practical exploitability.

Overall, this work provides an empirical characterization of the gap between theoretical cache leakage and practical key recovery on RISC-V platforms, and clarifies the conditions under which access-driven cache attacks remain effective in practice.

1 Introduction and Project Context

This report investigates a cache-based timing side-channel attack to extract partial key-dependent information from a T-table-based AES implementation. In particular, the attack targets first-round leakage to infer the most significant four bits (high nibble) of individual AES key bytes, rather than performing full secret key recovery. The attack was conducted on real hardware, specifically on a BeagleV Fire (Beagle Five Fire) platform, which was used as the target system for the experimental analysis.

The main objective of the project was not to design a novel attack from scratch, but rather to replicate and adapt an attack described in the existing literature to a specific hardware architecture. In particular, the goal was to evaluate whether the theoretical assumptions underlying the reference attack remain valid when deployed on real hardware, under realistic architectural constraints and non-ideal execution conditions.

The cache-timing measurement framework employed in this work was first developed and validated independently of any cryptographic workload. This preliminary phase established the feasibility of observing L1 cache state persistence across context switches, reliable cycle-level timing measurements, and controlled attacker-victim synchronization on the target platform. The AES attack presented in this report directly builds upon these validated properties.

Unlike purely simulated or idealized environments, real systems introduce a wide range of sources of noise and interference, including operating system activity, scheduling effects, and microarchitectural optimizations such as hardware prefetching. A significant part of this

project is therefore devoted to identifying, characterizing, and mitigating these sources of noise in order to preserve the information leakage exploited by the side channel.

The attack targets an AES implementation that relies on lookup tables (T-tables) to accelerate encryption. While this design choice improves performance, it introduces data-dependent memory accesses that can be exploited through cache timing analysis. By observing timing variations induced by cache contention, the attacker can probabilistically infer information about the secret key used by the victim process.

The experimental approach adopted in this work is incremental. The work progressed in two stages: an initial phase focused on the development and validation of a Prime+Probe measurement framework on real RISC-V hardware, followed by a second phase applying this framework to a cryptographic case study based on a T-table AES implementation. The project starts by verifying the existence of measurable information leakage at a coarse granularity, and progressively refines the attack until it becomes possible to predict the victim's access to individual cache lines with high accuracy. The final goal is to leverage this information to recover a significant portion of the AES key, limited to the first round of the algorithm.

It is important to emphasize that the scope of this project is intentionally constrained. The attack is evaluated under a controlled threat model, in which the attacker has partial control over synchronization and input data. While such assumptions may not hold in all deployment scenarios, they enable a precise and systematic analysis of the underlying microarchitectural leakage mechanisms and their practical limitations.

2 Hardware Architecture Overview

The experimental evaluation was conducted on a BeagleV Fire (Beagle Five Fire) platform built around a Microchip PolarFire SoC device. The PolarFire SoC integrates a RISC-V microprocessor subsystem and FPGA fabric within a single chip, providing a tightly coupled environment for both general-purpose computation and hardware acceleration.

The microprocessor subsystem includes a five-core CPU complex based on the U54-MC family, composed of four U54 application cores and one E51 monitor core. The U54 application cores implement the RV64GC instruction set and are designed to support rich operating systems such as Linux, while the E51 core implements RV64IMAC and is primarily used for boot, initialization, and system supervision. In the context of this project, only the U54 application cores are relevant, as both the attacker and the victim execute as user-space processes on these cores.

From a microarchitectural perspective, both U54 and E51 cores are single-issue, in-order processors employing a classic five-stage pipeline consisting of instruction fetch, decode and register fetch, execution, memory access, and write-back. The absence of out-of-order execution reduces some forms of timing variability compared to high-end OoO processors, but does not eliminate microarchitectural leakage arising from cache hierarchies, prediction mechanisms, or shared memory resources. As a result, cache-based timing side channels remain observable and exploitable.

The PolarFire SoC exposes a multi-level memory hierarchy composed of private L1 caches, a shared L2 memory subsystem, on-chip boot memory, and off-chip DRAM. Each U54 core has a private Level-1 instruction and data cache of 32 KB, 8-way set-associative, with a cache line size of 64 bytes, as specified in the PolarFire SoC documentation [2]. The L1 data cache follows a write-back policy and is implemented as a virtually indexed, physically tagged (VIPT) structure.

The L1 data cache is kept coherent through a directory-based coherence mechanism implemented in the shared L2 subsystem.

Above the private L1 caches, the system provides a shared Level-2 memory subsystem with a total capacity of 2 MB and support for error-correcting codes. The L2 structure is banked to increase bandwidth and internally organized as a set-associative cache. A distinctive feature of the PolarFire SoC is that the L2 subsystem can be reconfigured at runtime to operate either as a conventional cache, as deterministic memory regions or a coherent scratchpad.

The choice to target the private L1 data cache is not arbitrary. As experimentally verified during the preliminary framework evaluation, L1 cache state persists across context switches on the PolarFire SoC, enabling temporal sharing between attacker and victim when time-multiplexed on the same core. While a similar analysis could in principle be conducted at the shared L2 cache level, such an approach would require accounting for additional sources of interference, including instruction fetch activity and coherence traffic from other cores. For this reason, and due to time constraints, the present work focuses exclusively on L1 data cache effects.

The most relevant architectural properties for the present work can be summarized as follows:

- L1 caches are private to each core and are not spatially shared across different cores.
- The L2 subsystem is shared and enforces coherence across the application cores.

The private nature of the L1 caches has direct implications for the attack model considered in this project. Cache-based side-channel attacks targeting the L1 cache cannot rely on concurrent access by attacker and victim running on different cores. Instead, they require both processes to be scheduled on the same core and to access the cache at different points in time. This temporal sharing through time multiplexing is explicitly exploited in the experimental setup and constitutes a fundamental assumption of the threat model.

In addition to the cache hierarchy, other microarchitectural components may contribute to execution-time variability. Branch prediction mechanisms introduce speculative behavior that improves performance but can affect timing determinism; while not exploited as a primary leakage source in this work, they represent a secondary source of noise in fine-grained measurements.

Similarly, the presence and behavior of hardware data prefetchers on the target platform cannot be reliably inferred from the available experimental data. Nevertheless, to avoid potential interference from undocumented or implementation-specific prefetch mechanisms,

the attacker implementation adopts data-dependent and irregular memory access patterns as a precautionary measure.

It is worth noting that the PolarFire SoC microarchitecture is explicitly described as not being vulnerable to Meltdown and Spectre attacks, due to its simple in-order, single-issue pipeline and the absence of aggressive speculative execution mechanisms. However, this property does not imply immunity to cache-based side-channel attacks. In particular, the cache attack considered in this work does not rely on speculative execution or privilege bypass, but solely on data-dependent memory accesses and cache contention. As a result, even architectures that are not affected by Meltdown or Spectre remain vulnerable when cryptographic implementations employ lookup-table-based designs, such as AES T-table implementations.

3 Background on Cache-Based Side-Channel Attacks

Cache-based side-channel attacks exploit the fact that memory access latency depends on the state of the cache hierarchy. Cache hits are served with significantly lower latency than cache misses, which must be handled by lower levels of the memory system. This timing variability can be externally observed and exploited as a side channel.

Such attacks typically assume that an attacker and a victim share some portion of the cache hierarchy, either spatially or temporally. Spatial sharing occurs when multiple cores access a physically shared cache, such as a last-level cache, while temporal sharing arises when attacker and victim are time-multiplexed on the same core and access a private cache at different points in time.

A commonly studied target for cache-based attacks is software implementations of symmetric cryptography. In particular, optimized implementations of the Advanced Encryption Standard (AES) often rely on lookup tables to accelerate encryption. While such implementations improve performance, they introduce secret-dependent memory access patterns that directly couple cryptographic operations with cache behavior.

Early work by Bernstein demonstrated that timing variations caused by cache effects can leak information about AES secret keys [1]. Subsequent work by Osvik, Shamir, and Tromer [3] systematized these observations and introduced general cache attack techniques such as Prime+Probe and Evict+Time, along with a formal leakage model based on cache-line granularity.

A key insight of this model is that cache-based attacks do not typically reveal exact memory addresses or data values. Instead, the attacker observes effects at the granularity of cache lines, which group multiple memory locations together. As a consequence, the amount of information leaked by a cache-based side channel is fundamentally limited by the cache geometry and by the memory layout of the victim implementation.

4 AES T-table Implementation and Leakage Model

This section describes the AES implementation targeted by the attack and formalizes the leakage model exploited throughout the project. The analysis follows the access-driven cache attack framework introduced by Osvik, Shamir, and Tromer, and applies it to the specific cache geometry of the target platform. [3]

The victim implements AES encryption using precomputed lookup tables, commonly referred to as T-tables. In this design, the `SubBytes`, `ShiftRows`, and `MixColumns` operations are combined into table lookups to improve performance. Each T-table contains 256 entries, and each entry occupies 4 bytes, resulting in a total table size of 1024 bytes.

During the first round of AES, the table index used for each lookup is computed as:

$$\text{index} = P_i \oplus K_i$$

where P_i denotes a plaintext byte and K_i denotes the corresponding key byte. As a result, memory accesses performed by the victim during the first round are directly dependent on secret key material. In a Prime+Probe attack, the attacker does not observe exact memory addresses or table indices. Instead, it can only determine whether a given cache line has been accessed by the victim. Consequently, the granularity of the leakage is determined by the cache line size rather than by the size of individual table entries.

On the target platform, the L1 data cache uses a cache line size of 64 bytes. Since each T-table entry occupies 4 bytes, a single cache line contains:

$$\frac{64}{4} = 16$$

contiguous table entries. Cache observations therefore distinguish only between groups of 16 consecutive indices. Formally, the observable quantity is not the index itself, but:

$$\left\lfloor \frac{\text{index}}{16} \right\rfloor = \text{index} \gg 4$$

which corresponds to the upper four bits of the table index.

Substituting the AES first-round access pattern, the observable cache-line index becomes:

$$(P_i \oplus K_i) \gg 4$$

Because the plaintext is known to the attacker, observing the accessed cache line allows inference of:

$$K_i \gg 4$$

that is, the most significant four bits (high nibble) of the AES key byte. The remaining lower four bits are mapped to different entries within the same cache line and therefore cannot be distinguished at cache-line granularity.

This limitation is structural and follows directly from the cache geometry and the use of T-table-based AES, rather than from the specific attack implementation adopted in this

project. While a single-round observation is inherently limited to recover only the high nibble of each key byte, additional key information can be inferred by aggregating leakage across multiple AES rounds, as in second-round attacks that exploit inter-byte mixing to introduce cross-round constraints.

5 Threat Model and Experimental Assumptions

The threat model adopted in this project is intentionally constrained in order to enable a controlled and reproducible analysis of cache-based information leakage on real hardware. Rather than attempting to model a fully general adversarial scenario, the focus is on isolating the microarchitectural effects exploited by the attack and on evaluating their impact under well-defined experimental conditions.

The attacker is an unprivileged user-space process executing on the same system as the victim. The attacker has no direct access to the victim's memory, control flow, or secret key material, and does not rely on elevated privileges, access to physical memory, or shared cryptographic state. The only observable channel available to the attacker is execution time, inferred indirectly through its own memory accesses.

The threat model assumes that the attacker and the victim are time-multiplexed on the same physical core, thereby sharing the private L1 data cache. Since the L1 cache is private to each core and not spatially shared across cores [2], temporal sharing is required to make L1-based cache timing attacks feasible. This sharing is enforced through explicit synchronization, ensuring alternating attacker and victim access phases.

The execution environment consists of a minimal Linux system, in which no additional user processes are scheduled beyond the baseline operating system activity. The attacker does not have root privileges and does not access privileged memory regions. Shared memory is used exclusively for synchronization and for providing plaintext inputs to the victim, and not for sharing internal cryptographic data structures or secret-dependent state.

The attacker is assumed to have partial control over the victim's inputs. In particular, the plaintext processed by the AES implementation is known and/or chosen by the attacker, while the secret key remains fixed for the duration of the attack. These assumptions are realistic in many practical scenarios, as plaintext is often observable or controllable and cryptographic keys are typically reused for periods that are long compared to the execution time of a cache timing attack.

The attack is restricted to the first round of the AES algorithm. This assumption is primarily methodological rather than fully realistic, and is introduced to isolate the leakage mechanism induced by the T-table implementation while minimizing additional sources of noise. In a complete AES execution, later rounds introduce further memory accesses and inter-round dependencies that significantly complicate the attribution of observed cache effects to specific key-dependent operations. Restricting the analysis to the first round allows the attack to focus on the dominant and most directly observable source of leakage without altering the fundamental nature of the side channel.

An additional assumption in the threat model concerns the memory alignment of the victim's T-tables. In the considered implementation, the tables are page-aligned, meaning that their base address has a known page offset equal to zero. On the target architecture, the L1 data cache uses a set index that is entirely contained within the page offset. As a result, page alignment implies a deterministic mapping between cache lines of the T-tables and L1 cache sets, without requiring knowledge of the virtual-to-physical address translation.

This property reduces uncertainty in the attack by eliminating the need for an explicit discovery phase to locate the victim's tables in the cache. While it does not change the underlying side-channel mechanism, it significantly increases experimental complexity. Removing this assumption does not eliminate the underlying leakage mechanism, but significantly increases the complexity of the attack.

The core assumptions underlying the experimental setup can be summarized as follows:

- Attacker and victim are time-multiplexed on the same core, enabling temporal sharing of the private L1 data cache.
- The AES implementation uses page-aligned T-tables and does not employ constant-time countermeasures.
- The attacker can choose plaintext inputs and measure its own memory access times with sufficient resolution.
- The execution environment is minimally loaded, reducing uncontrolled interference.

The attack implemented in this project is explicitly designed to operate on the L1 data cache under the stated assumptions, including controlled scheduling and chosen plaintexts. If these assumptions are violated, the underlying side-channel principles remain valid, but the implemented attack no longer operates in its intended regime.

In such cases, the absence of reliable temporal sharing of the L1 cache prevents meaningful observations, and the experimental results do not converge. Addressing these scenarios would require targeting different microarchitectural resources, such as the shared L2 cache, and would therefore necessitate substantial modifications to the implementation.

Overall, the threat model defines a favorable but explicit experimental setting in which cache timing leakage can be reliably observed and analyzed.

6 Attack Methodology: Prime+Probe

The attack implemented in this project follows the Prime+Probe paradigm, an access-driven cache-based timing side-channel technique originally formalized by Osvik, Shamir, and Tromer [3], in which the attacker infers victim memory accesses by inducing and observing contention on shared cache resources.

Rather than observing the victim directly, the attacker infers information about the victim's execution by measuring variations in its own memory access times.

The attack targets the private L1 data cache and relies on temporal sharing between attacker and victim processes. Given that the L1 cache is not spatially shared across cores, the methodology explicitly assumes time multiplexing on a single core, as described in the threat model. Within this setting, cache state persists across context switches, allowing the attacker to observe the effects of the victim's memory accesses.

The Prime+Probe attack consists of three conceptual phases. During the *Prime* phase, the attacker fills all the cache sets with its own data, establishing a known cache state. The victim then executes its AES encryption routine, performing key-dependent memory accesses to the T-tables that may evict some of the attacker's cache lines. Finally, during the *Probe* phase, the attacker re-accesses its own data and measures the access latency to determine which cache sets have experienced eviction.

The effectiveness of this methodology relies on the fact that cache hits and cache misses exhibit measurably different access times. Accesses served from the L1 cache complete with low latency, whereas accesses that miss in L1 and are serviced by lower levels of the memory hierarchy incur significantly higher delays. By comparing measured access times against derived thresholds, the attacker can classify whether a given cache set was accessed by the victim.

To reduce measurement noise and avoid interference from hardware prefetching, the attacker employs irregular memory access patterns based on pointer chasing. This ensures that each memory access depends on the result of the previous one, preventing the hardware from predicting future accesses and preloading cache lines that would otherwise distort timing measurements.

The attack implementation follows a strict alternation between attacker and victim execution phases. Synchronization primitives are used to ensure that the Prime, Victim, and Probe phases occur in a controlled order, minimizing overlap and unintended cache pollution. This design choice emphasizes temporal separation rather than concurrency, simplifying the interpretation of observed cache effects.

At a high level, the methodology can be summarized as follows:

- The attacker primes selected L1 cache sets with its own data.
- The victim executes a configurable number of AES rounds, performing key-dependent table lookups.
- The attacker probes the same cache sets and classifies access times to infer victim activity.

7 Experimental Setup and Orchestration

7.1 Execution Environment and Experiment Orchestration

Before targeting the AES implementation, the experimental setup was validated through a dedicated cache-timing analysis framework designed to study L1 cache behavior on the target platform. This preliminary phase verified the persistence of cache state across context

switches, the reliability of cycle-level timing measurements, and the effectiveness of synchronization mechanisms under real operating system conditions.

All experiments were conducted on a minimal Linux system running on the target platform, with no additional user-level processes scheduled beyond baseline operating system activity. Both the attacker and the victim execute as independent user-space processes and are explicitly bound to the same physical core in order to enforce temporal sharing of the private L1 data cache.

While operating system preemption cannot be entirely eliminated, this setup ensures that attacker and victim repeatedly access the same cache instance, allowing cache state to persist across context switches. Synchronization between the two processes is implemented using POSIX named semaphores, enforcing a strict alternation pattern: when the attacker performs the Prime and Probe phases, the victim remains inactive, and vice versa. This kernel-mediated synchronization avoids concurrent cache access and simplifies the interpretation of timing measurements by establishing a well-defined execution order.

The overall execution and coordination of the experiment are managed through a dedicated Makefile, which acts as the central orchestration mechanism rather than merely a build script. The Makefile controls the full lifecycle of each experimental run, including process initialization, core affinity assignment, execution order, and cleanup. This approach ensures that experiments are performed under consistent conditions and reduces the likelihood of manual errors.

In particular, the Makefile is responsible for:

- compiling attacker and victim with consistent compiler flags and optimization levels,
- launching the processes in the correct order and binding them to the same core,
- storing results in the designated output directory,
- terminating processes and cleaning up shared state at the end of each run.

During the early stages of the project, this orchestration framework was used to explore different attack configurations. For example, parameters controlling the number of cache sets accessed by the victim were varied to study partial versus full cache eviction and to evaluate how measurement noise scales with the size of the monitored cache region. In the later stages, once relevant parameter ranges had been identified, the experiments were automated using Bash scripts to perform systematic parameter sweeps and to ensure reproducible execution of the final measurements.

7.2 Low-Level Measurement Infrastructure

Timing measurements are performed entirely from user space. The attacker measures the latency of its own memory accesses and uses these measurements as the sole source of information about the victim's behavior. No privileged instructions, kernel instrumentation, or hardware performance counters are used.

Reliable cache timing analysis on real hardware requires strict control over compiler behavior, instruction ordering, and microarchitectural side effects. To this end, cycle- accurate timing measurements are obtained using the `rdcycle` instruction. Explicit compiler barriers are inserted around timing boundaries using inline assembly with `memory clobbers`, preventing the compiler from reordering memory operations across measurement regions during optimization.

In addition to compiler-level barriers, architectural memory ordering is enforced using the `fence` instruction. Memory fences are placed before and after timed memory access sequences to prevent reordering of loads and stores at the microarchitectural level, ensuring that observed timing variations reflect cache behavior rather than relaxed memory consistency effects.

All code involved in timing measurements is compiled with optimization enabled (-O2) to preserve realistic instruction scheduling and register allocation. At the same time, specific compiler optimizations known to interfere with fine-grained timing analysis—such as function inlining, loop unrolling, and automatic vectorization—are explicitly disabled. This allows the generated code to retain performance-relevant characteristics while avoiding transformations that would obscure the structure and timing of the measured code paths. Additional care is taken to enforce explicit data dependencies and memory accesses where needed, ensuring stable and interpretable timing measurements.

The measurement infrastructure was validated independently of the cryptographic workload to confirm cache-set observability, timing stability, and resistance to common sources of noise such as speculative execution, hardware prefetching, and background system activity. The same infrastructure was subsequently reused without modification for the AES attack presented in this work.

8 Noise Sources and Mitigation Techniques

Cache-based timing attacks on real hardware are inherently affected by multiple sources of noise. In this work, these sources were identified empirically through preliminary cache-timing experiments conducted prior to the cryptographic attack, rather than through purely theoretical analysis. The resulting mitigation strategies were designed and validated in this exploratory phase, ensuring that they preserve the underlying leakage mechanism while improving the interpretability of the observed timing measurements.

One major source of noise is operating system activity. Context switches, scheduler decisions, and background kernel tasks can evict cache lines or delay execution in ways that are unrelated to the victim's memory accesses. Even in a minimal Linux environment, such effects cannot be completely eliminated and manifest as sporadic high-latency outliers in timing measurements.

Synchronization between attacker and victim represents a second, more subtle source of interference. Semaphores are required to enforce time multiplexing and strict alternation between execution phases, but they also introduce cache pollution by accessing shared memory and kernel data structures. As a result, synchronization primitives simultaneously enable the

attack and degrade measurement quality, creating an inherent trade-off between control and noise.

Additional noise is introduced by microarchitectural features designed to improve performance. Hardware mechanisms such as data prefetching can introduce cache fills and evictions that are not causally related to the victim's execution. In particular, regular or sequential memory access patterns in the attacker can trigger prefetching, effectively masking or distorting the cache effects induced by the victim.

Several mitigation techniques were adopted to reduce the impact of these noise sources. To limit interference from the operating system, experiments were conducted in a minimally loaded environment and repeated over a large number of trials, allowing statistically consistent effects to dominate transient disturbances. Measurements corresponding to extreme delays caused by preemption or unrelated cache activity were identified as outliers and excluded during data analysis.

To mitigate the effects of hardware prefetching, the attacker avoids predictable memory access patterns. Instead of sequential or strided accesses, the probing phase employs pointer-chasing techniques, in which each memory access depends on the value retrieved by the previous one. This data-dependent access pattern prevents the prefetcher from anticipating future accesses and significantly improves the stability of timing measurements.

Timing measurements exhibit heavy-tailed and asymmetric distributions, with outliers occurring almost exclusively on the high-latency side due to cache misses, preemption, or unrelated system activity. In this context, the mean is a poor estimator, as it is disproportionately influenced by rare but extreme delays. The median provides a more robust representation of typical access latency and is therefore used throughout the analysis. Aggregating measurements across multiple executions reduces variance but does not introduce new information; it merely increases confidence in the observed tendencies.

Despite these mitigation strategies, noise cannot be entirely eliminated. The attack remains probabilistic, and reliable inference requires aggregating observations across many independent executions. By explicitly accounting for noise sources and designing the experiment to minimize their impact, the attack achieves sufficient signal quality to enabling meaningful inference of high-nibble key information in the controlled setting considered.

9 Classification Strategy and Data Analysis

The Prime+Probe attack produces timing measurements that are too noisy to be interpreted on a per-sample basis. Reliable key inference therefore requires a classification strategy that is robust to noise and capable of distinguishing cache hits from cache misses using statistical aggregation.

At a conceptual level, the attack relies on a causal chain linking known plaintexts to key-dependent cache behavior. For each encryption, the plaintext determines the indices accessed in the AES T-tables during the first round, which in turn map to specific cache lines and cache sets. The victim's execution modifies the cache state, and the attacker observes these

modifications through probing-time measurements. Cache sets are classified as accessed or not accessed, and these classifications are aggregated across repeated executions to evaluate key hypotheses.

The classification pipeline adopted in this work is fully data-driven. Rather than relying on fixed thresholds or architecture-specific timing constants, all decision boundaries are derived empirically from the observed timing distributions. This approach allows the classifier to adapt to variability across cache sets and across experimental runs.

The analysis is organized into three phases: calibration, training, and testing.

During the calibration phase, an initial set of measurements is collected to characterize baseline cache behavior. Timing measurements exhibit heavy-tailed and asymmetric distributions, with outliers occurring almost exclusively on the high-latency side due to preemption or unrelated system activity. In this context, the mean is a poor estimator, as it is disproportionately influenced by rare but extreme delays. The median provides a more robust representation of typical access latency and is therefore used throughout the analysis. Measurements exceeding a multiple of the median are treated as outliers and excluded, as they do not carry useful information about the victim's execution.

In the training phase, timing measurements are collected under controlled conditions, both in the presence and in the absence of victim accesses to the monitored cache sets. For each cache set, separate timing distributions are obtained for access and no-access cases. A set-specific classification threshold is then defined as an intermediate value between the medians of these two distributions, yielding a decision boundary tailored to the observed behavior of each cache set.

The testing phase applies the learned thresholds to new measurements collected while the victim processes different plaintexts. For each execution, cache sets are classified as accessed or not accessed based on the observed probe times. These binary observations are aggregated across many independent executions to infer key-dependent access patterns.

Several alternative classification strategies were explored. In particular, repeated probing of the same cache set within a single execution was initially considered as a means of improving separability. While timing differences may remain observable across successive probes, these measurements are strongly correlated. The first probe already resolves the cache state left by the victim, and subsequent accesses increasingly reflect a cache state dominated by the attacker's own activity. As a result, repeated probing does not provide independent information and does not improve classification accuracy.

Some cache sets exhibit timing distributions for access and no-access cases that overlap significantly under any reasonable threshold. Such behavior indicates intrinsic non-separability rather than insufficient sampling. These cache sets are therefore classified as non-informative and excluded from further analysis to avoid degrading overall classification quality.

Repeated executions of the victim do not correspond to adaptive behavior or oracle strengthening. They serve solely as a means of statistical aggregation, allowing noise to be averaged out while preserving the underlying leakage signal. Each execution is independent and does not alter the victim's behavior in response to previous observations.

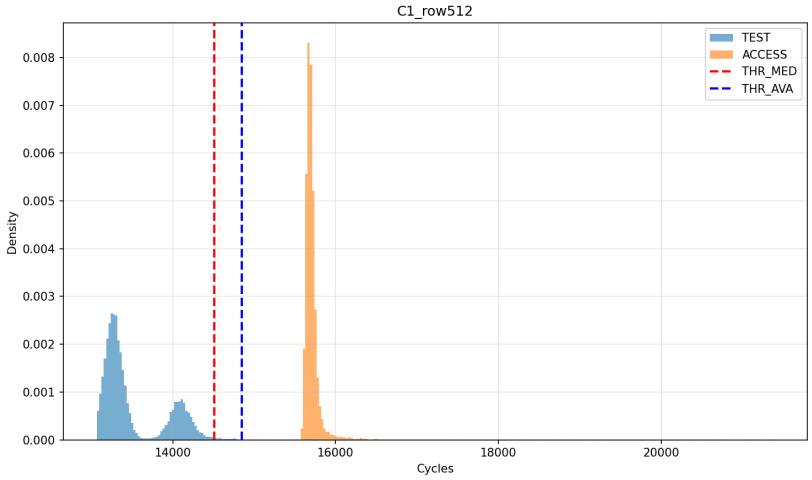

Figure 1: Probe latency distributions for a large victim footprint (512 rows) without pointer chasing.

10 Experimental Results

This section presents the experimental results obtained by applying the Prime+Probe attack under the assumptions described in the threat model. The goal of the evaluation is not only to assess whether cache timing leakage exists, but also to determine to what extent this leakage can be exploited for cache-set prediction and, ultimately, for key inference.

It is important to distinguish between three conceptually separate stages of the attack: the existence of leakage, the ability to exploit this leakage to classify cache-set accesses, and the successful inference of key-dependent information. While these stages are causally related, they introduce progressively stronger assumptions and exhibit different success metrics. The presence of measurable timing differences does not automatically imply reliable cache-set classification, nor does correct classification guarantee accurate partial key recovery.

Leakage Observability. Leakage observability is first evaluated by directly inspecting probe latency distributions, shown in Fig. **??** and Fig. **??**. When the victim accesses a large memory footprint (512 rows), the latency distributions corresponding to accessed and non-accessed cache sets are clearly separated. In this configuration, cache sets evicted by the victim consistently exhibit higher probe latencies, indicating strong and easily observable cache interference. When the number of victim accesses is reduced to a small footprint (8 rows), the two distributions progressively overlap and become difficult to distinguish.

In this low-leakage regime, threshold selection becomes critical. High-latency outliers significantly influence thresholds derived from the arithmetic mean. As a result, the decision boundary is shifted toward higher values, increasing the rate of misclassification. Although pointer chasing is effective at stabilizing individual probe measurements, it does not introduce clear structural changes in the raw latency distributions. Figure **??** reports the probe latency distributions obtained with pointer chasing enabled.

To quantify these observations, multiple probing and aggregation configurations are com-

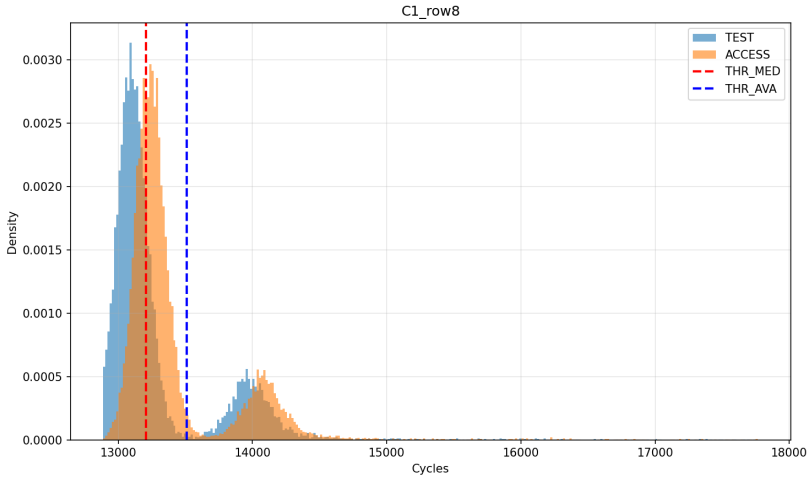

Figure 2: Probe latency distributions for a small victim footprint (8 rows) without pointer chasing.

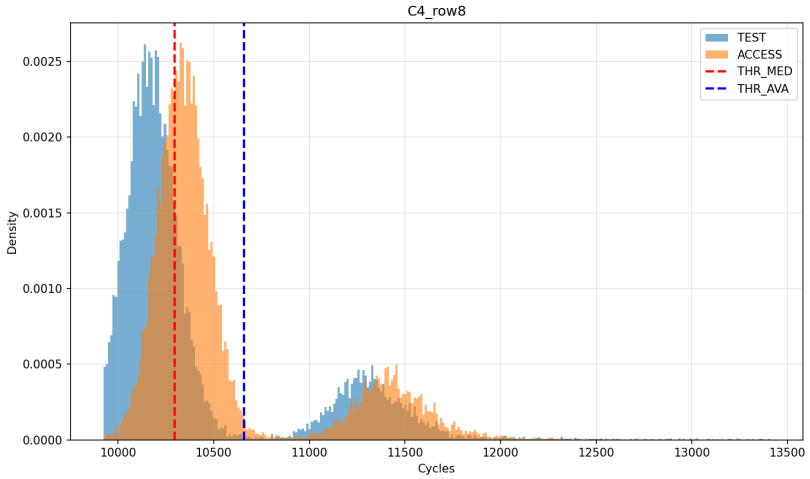

Figure 3: Probe latency distributions for a small victim footprint (8 rows) with pointer chasing enabled.

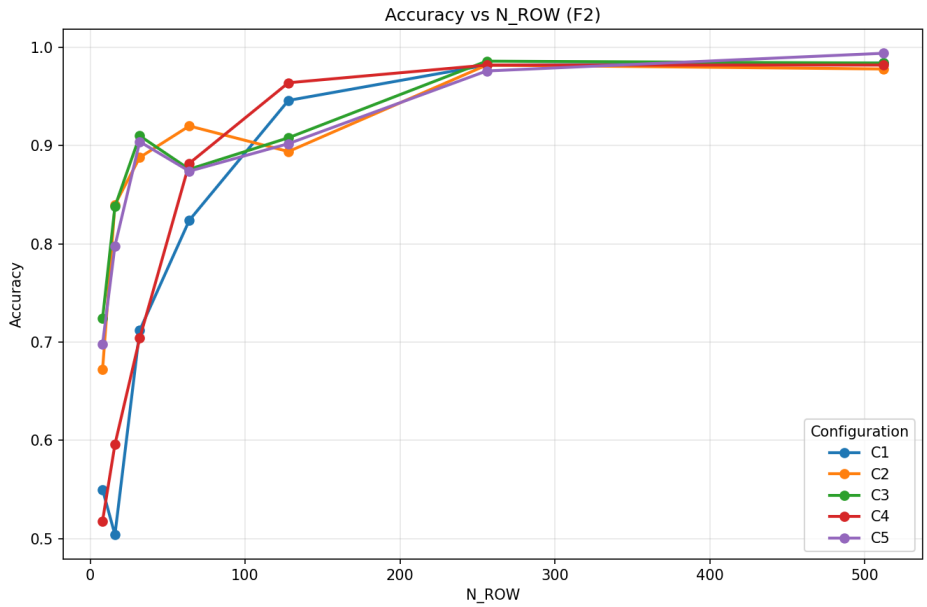

Figure 4: Classification accuracy as a function of the victim footprint for all evaluated configurations (C1–C5).

pared by measuring classification accuracy as a function of the victim footprint, as shown in Fig. 4. The evaluated configurations include five fixed probing and aggregation setups:

- **C1**: baseline indexed probing with mean-based aggregation and no calibration;
- **C2**: indexed probing with median-based aggregation and no calibration;
- **C3**: pointer-chasing probing with median-based aggregation and no calibration;
- **C4**: pointer-chasing probing with mean-based aggregation and no calibration;
- **C5**: pointer-chasing probing with median-based aggregation and calibration-enabled outlier rejection.

The accuracy curves show that median-based aggregation consistently outperforms mean-based aggregation when the hit and miss distributions are close. This improvement comes at the cost of a slight accuracy penalty when the distributions are only partially overlapped, where the mean provides a sharper decision boundary. Pointer chasing yields a modest improvement across configurations, indicating improved robustness to noise rather than increased leakage strength.

Finally, leakage observability is examined as a function of the number of probe iterations used during training. Fig. 5 reports classification accuracy as the number of iterations increases for a fixed victim footprint. Beyond a moderate number of iterations, no substantial accuracy improvement is observed. This behavior indicates diminishing returns from further probing and suggests that residual errors are dominated by irreducible system noise rather

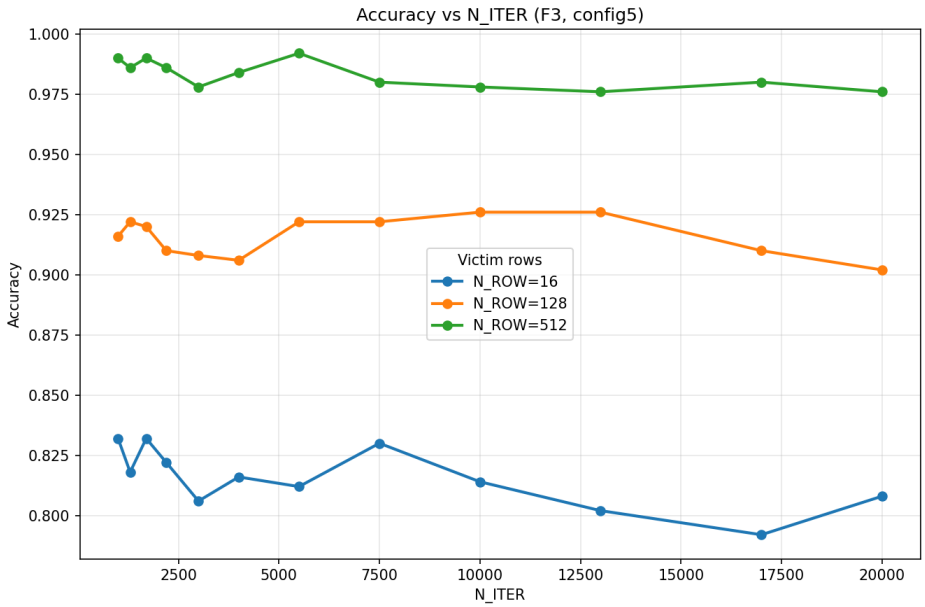

Figure 5: Classification accuracy as a function of the number of probe iterations for a fixed victim footprint.

than insufficient training length. Increasing the number of iterations therefore cannot compensate for weak leakage conditions induced by small victim footprints.

Per-Set Cache-Set Classification. Given observable leakage, the next step is to evaluate the attacker's ability to classify cache-set accesses at per-set granularity. Using data-driven thresholds derived during the training phase, each cache set is classified as accessed or not accessed by the victim. While this approach yields high accuracy for a subset of cache sets, others consistently exhibit overlapping timing distributions and are therefore excluded from further analysis.

A detailed per-set analysis reveals that probe latency behavior is strongly cache-set dependent. When latency distributions are examined independently for each set, substantial variability emerges across the cache, motivating a per-set characterization rather than a global classification strategy. Figure 6 shows the latency distributions for a representative subset of eight cache sets. While some sets exhibit well-separated hit and miss distributions, others show substantial overlap. This noise is largely attributable to persistent interference sources such as operating system activity, stack accesses, shared runtime data structures, and fixed memory mappings unrelated to the victim. As a result, leakage quality is not uniform across the cache.

Based on these observations, a dedicated threshold is computed independently for each cache set. Table 1 reports the thresholds derived during the training phase for all cache sets. Sets whose distributions remain indistinguishable after thresholding are marked as non-informative and excluded from subsequent prediction stages. This per-set filtering prevents

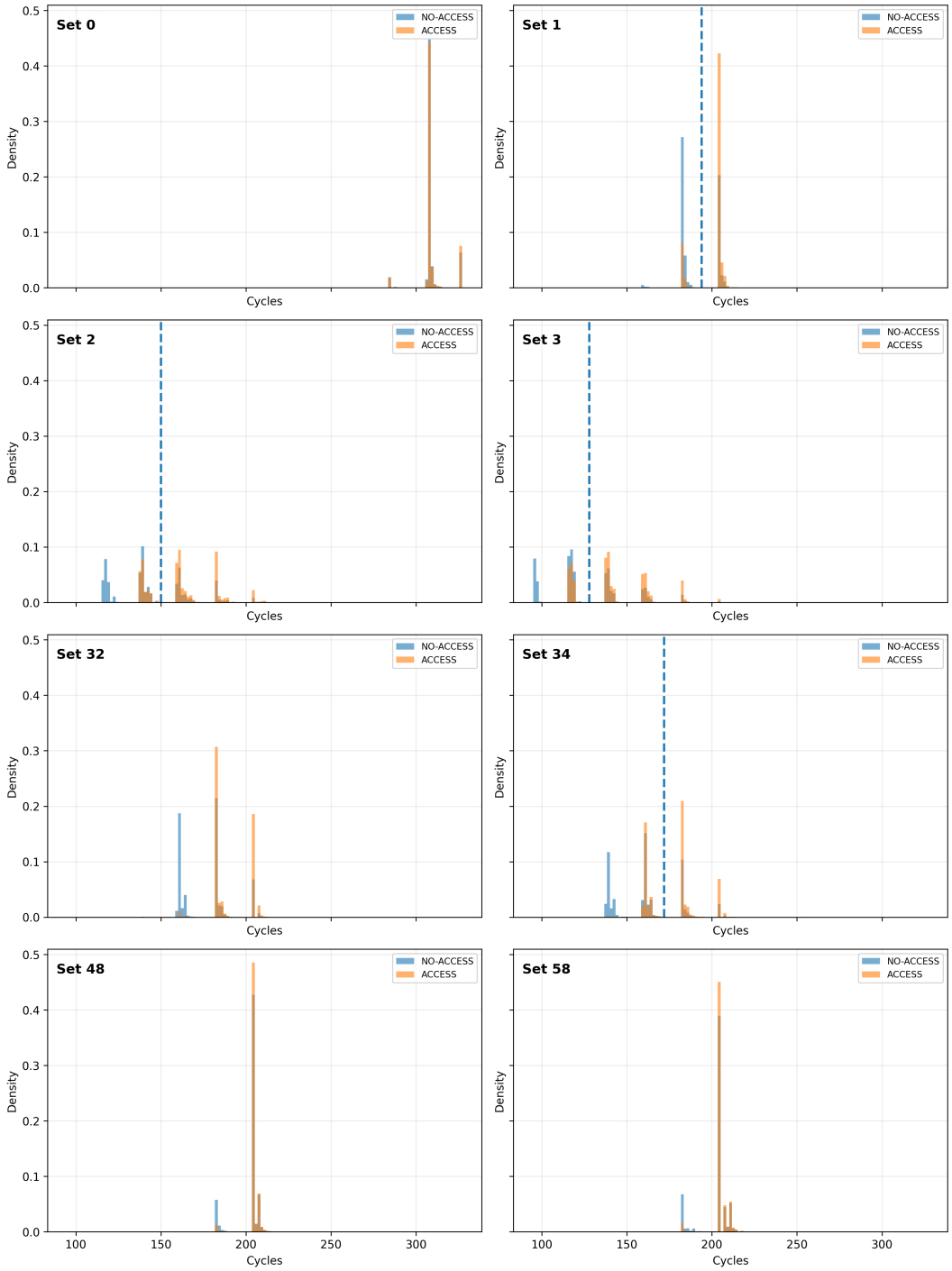

Figure 6: Probe latency distributions for eight representative cache sets.

Table 1: Per-set latency statistics and threshold derivation results.

Set	Miss	Hit	Gap	Status	Set	Miss	Hit	Gap	Status
0	307	307	0	Invalid	32	183	183	0	Invalid
1	184	204	20	Valid	33	161	183	22	Valid
2	139	161	22	Valid	34	161	183	22	Valid
3	117	139	22	Valid	35	183	205	22	Valid
4	160	161	1	Valid	36	139	161	22	Valid
5	166	204	38	Valid	37	95	117	22	Valid
6	163	183	20	Valid	38	139	161	22	Valid
7	161	182	21	Valid	39	183	184	1	Valid
8	138	144	6	Valid	40	161	183	22	Valid
9	183	189	6	Valid	41	185	205	20	Valid
10	161	183	22	Valid	42	186	205	19	Valid
11	139	142	3	Valid	43	185	205	20	Valid
12	117	139	22	Valid	44	161	183	22	Valid
13	117	139	22	Valid	45	161	183	22	Valid
14	73	95	22	Valid	46	161	183	22	Valid
15	95	117	22	Valid	47	117	139	22	Valid
16	117	139	22	Valid	48	205	205	0	Invalid
17	117	139	22	Valid	49	205	205	0	Invalid
18	95	117	22	Valid	50	205	205	0	Invalid
19	139	161	22	Valid	51	205	205	0	Invalid
20	139	161	22	Valid	52	205	205	0	Invalid
21	139	161	22	Valid	53	205	205	0	Invalid
22	139	161	22	Valid	54	183	205	22	Valid
23	95	117	22	Valid	55	205	205	0	Invalid
24	95	117	22	Valid	56	205	205	0	Invalid
25	72	95	23	Valid	57	205	205	0	Invalid
26	161	182	21	Valid	58	205	205	0	Invalid
27	95	117	22	Valid	59	161	183	22	Valid
28	161	182	21	Valid	60	184	205	21	Valid
29	161	183	22	Valid	61	183	205	22	Valid
30	139	160	21	Valid	62	183	205	22	Valid
31	184	205	21	Valid	63	205	205	0	Invalid

noisy cache regions from degrading overall classification accuracy.

Per-set predictions are then used as input to the cache-set inference stage. For each trial, multiple Prime+Probe measurements are aggregated, and cache sets are ranked according to the number of detected accesses. The sets with the highest access counts are selected as the most likely victim-accessed sets. Figure 7 reports a representative per-set ranking obtained from aggregated Prime+Probe measurements. Cache sets are ordered according to the number of detected accesses, and the set with the highest count is selected as the predicted victim-accessed set. It is interesting to highlight that certain cache sets are recurrently present in the rankings and appear in cases of misclassifications. These systematic errors suggest the presence of persistent noise sources and indicate clear opportunities for further optimization, such as adaptive weighting or dynamic set exclusion.

```
#1 | SET 12 | Hits: 62
#2 | SET 36 | Hits: 62
#3 | SET 08 | Hits: 52
#4 | SET 07 | Hits: 50
#5 | SET 11 | Hits: 49
----------------------
PREDICTION: Victim is in SET 12
CORRECT set is: 36
```

```
#1 | SET 45 | Hits: 70
#2 | SET 12 | Hits: 62
#3 | SET 07 | Hits: 51
#4 | SET 08 | Hits: 45
#5 | SET 11 | Hits: 44
----------------------
PREDICTION: Victim is in SET 45
CORRECT set is: 45
```

```
#1 | SET 15 | Hits: 61
#2 | SET 12 | Hits: 59
#3 | SET 08 | Hits: 45
#4 | SET 13 | Hits: 44
#5 | SET 39 | Hits: 44
----------------------
PREDICTION: Victim is in SET 15
CORRECT set is: 15
```

```
#1 | SET 12 | Hits: 59
#2 | SET 07 | Hits: 50
#3 | SET 33 | Hits: 47
#4 | SET 11 | Hits: 45
#5 | SET 13 | Hits: 42
----------------------
PREDICTION: Victim is in SET 12
CORRECT set is: 60
```

```
#1 | SET 13 | Hits: 75
#2 | SET 12 | Hits: 61
#3 | SET 08 | Hits: 52
#4 | SET 07 | Hits: 50
#5 | SET 36 | Hits: 48
----------------------
PREDICTION: Victim is in SET 13
CORRECT set is: 13
```

```
#1 | SET 24 | Hits: 68
#2 | SET 12 | Hits: 61
#3 | SET 11 | Hits: 50
#4 | SET 07 | Hits: 49
#5 | SET 43 | Hits: 48
----------------------
PREDICTION: Victim is in SET 24
CORRECT set is: 24
```

Figure 7: Representative per-set ranking based on aggregated access counts.

Finally, the effect of aggregation is evaluated by increasing the number of Prime+Probe measurements combined within each trial. Figure 8 shows classification accuracy as a function of the aggregation factor. As the number of aggregated measurements increases, prediction accuracy improves monotonically, confirming that aggregation effectively suppresses random noise. However, accuracy gains eventually diminish, indicating that aggregation cannot fully compensate for structurally noisy cache sets.

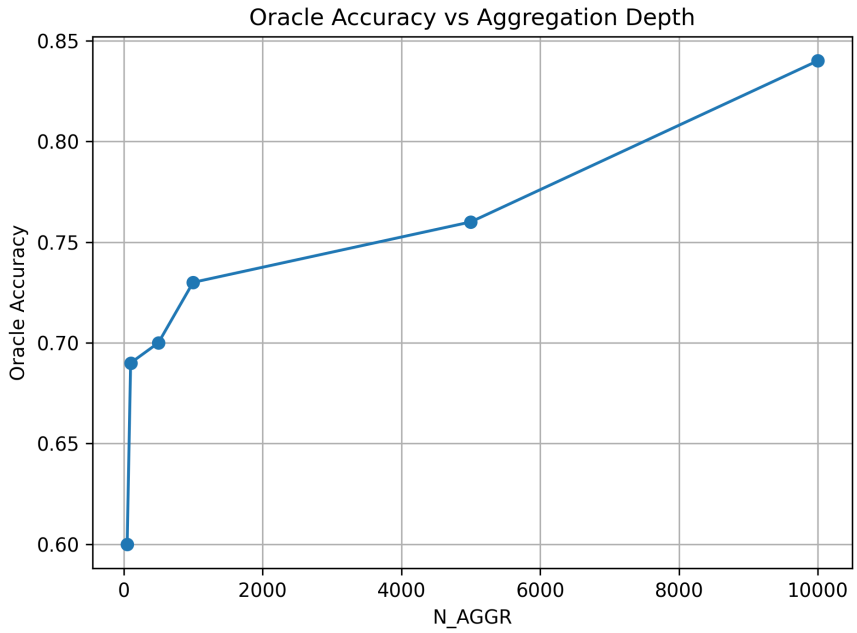

Figure 8: Classification accuracy as a function of the number of aggregated Prime+Probe measurements per trial.

Key Inference. The final stage of the attack aggregates cache-set classifications across multiple victim executions in order to infer key-dependent access patterns. For each key byte, candidate values are ranked according to their consistency with the observed cache behavior.

When the victim executes a single AES round, the attack achieves 100% accuracy on the high nibbles of the secret key. In this configuration, a complete key inference run requires a single victim execution and completes in approximately 2 minutes and 52 seconds. This result confirms that, under minimal diffusion, cache-set leakage directly reflects first-round table accesses and can be exploited with high confidence.

Increasing the number of AES rounds has a negative impact on inference accuracy, as shown in Fig. 9. As additional rounds are executed, memory accesses corresponding to different key bytes and rounds overlap in time, reducing the distinguishability of key-dependent access patterns. As a consequence, the ranking of key candidates becomes less reliable and overall accuracy degrades.

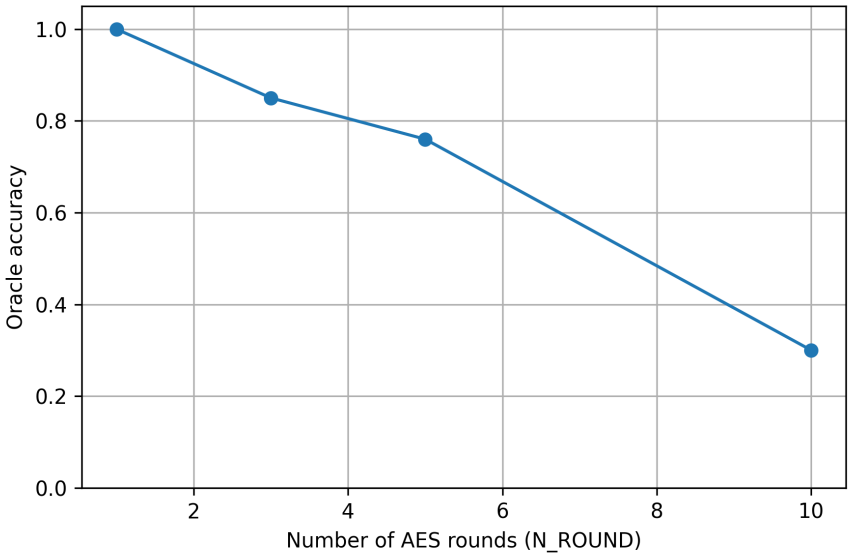

Figure 9: Key inference accuracy as a function of the number of AES rounds executed by the victim.

Due to the duration of the experiment, accuracy evaluation for multi-round configurations was performed over a limited number of trials. In particular, each configuration was tested on ten independent runs. While this limits statistical confidence, the observed trend is consistent across all measurements and clearly highlights the impact of increased diffusion on cache-based key inference.

Interpretation of Results. Taken together, the experimental results show that cache timing leakage on the target platform is clearly observable and can be exploited in a controlled attack setting. However, the evaluation also highlights that exploitability emerges only through a sequence of increasingly selective and noise-aware processing stages. While raw timing measurements reveal the presence of leakage, reliable cache-set classification requires per-set characterization, thresholding, and the exclusion of structurally noisy cache regions.

The per-set analysis demonstrates that cache behavior is highly non-uniform: only a subset of cache sets provides stable and informative leakage, while others are persistently affected by

interference unrelated to the victim. Aggregation across multiple Prime+Probe measurements mitigates random noise and improves prediction accuracy, but cannot fully compensate for sets that exhibit intrinsically overlapping timing distributions. As a result, classification accuracy is ultimately bounded by structural noise sources rather than insufficient sampling.

Key inference further amplifies these limitations. Perfect recovery of key-dependent information is achieved only under minimal diffusion, when a single AES round directly exposes first-round table accesses. As the number of rounds increases, memory access patterns overlap and diffuse, reducing the reliability of cache-based observations and degrading inference accuracy. These results emphasize the distinction between the existence of a side-channel and its practical effectiveness, showing that successful exploitation depends critically on architectural properties, execution control, and careful statistical filtering rather than on leakage presence alone.

11 Limitations and Discussion

While the experimental results demonstrate that cache-based timing leakage can be exploited to recover information about an AES key, the attack presented in this project is subject to several important limitations. These limitations stem both from the threat model assumptions and from design choices made to control noise and isolate the leakage mechanism. These limitations are consistent with the behavior observed during the preliminary cache-timing analysis, where reduced victim memory footprints and increased system activity led to overlapping timing distributions and loss of classification reliability.

A primary limitation concerns the reliance on L1 cache attacks under controlled scheduling conditions. The attack is explicitly dimensioned for the private L1 data cache and assumes temporal sharing between attacker and victim through time multiplexing on the same core. If this assumption does not hold, the implemented attack does not degrade gracefully: meaningful observations cannot be obtained, and the experimental results fail to converge. Although the underlying side-channel principles remain valid, exploiting them in such scenarios would require targeting different microarchitectural resources, such as the shared L2 cache, and would necessitate substantial changes to the implementation.

A second limitation is the restriction of the analysis to the first round of the AES algorithm. This choice is primarily methodological and was introduced to reduce noise and simplify attribution between observed cache effects and key-dependent memory accesses. While this approach preserves the fundamental leakage mechanism, it limits the realism of the attack, as a complete AES execution involves multiple rounds with additional memory accesses and interactions that are not captured in this model.

The attack also assumes partial control over the victim's inputs and a fixed secret key for the duration of the experiment. Although these assumptions are reasonable in many practical contexts, they do not hold universally and contribute to making the experimental setting more favorable than a fully uncontrolled environment.

Several architectural and system-level factors further constrain the generality of the results.

In particular, the page-aligned layout of the T-tables enables deterministic reasoning about cache set usage; without this property, the attacker would be required to perform an additional discovery phase to identify the cache sets associated with the victim's data. Similarly, the use of a minimal Linux environment reduces background noise and uncontrolled interference. Under higher system load, increased scheduling activity and cache pollution would likely degrade classification accuracy and reduce the reliability of the attack.

Despite these limitations, the results remain significant from a security perspective. They show that cache timing side channels can be exploited on real hardware when architectural and software conditions are favorable. Moreover, the need for controlled conditions highlights an important trade-off: optimizations aimed at performance and predictability, such as aligned data structures and cache-friendly layouts, can inadvertently simplify side-channel exploitation.

From a defensive perspective, several countermeasures can be employed to mitigate the type of cache-based leakage demonstrated in this project. At the software level, the most effective approach is the use of constant-time cryptographic implementations that avoid secret-dependent memory accesses altogether, thereby eliminating the root cause of the side channel.

At the system level, reducing or eliminating cache sharing between security domains can significantly limit the attack surface. This can be achieved through core isolation, cache partitioning, or scheduling policies that prevent untrusted processes from time-multiplexing the same private cache resources as sensitive workloads. While such techniques are effective, they often incur performance or flexibility costs and may not be practical in all deployment scenarios.

12 Conclusions and Future Work

This project investigated the practical feasibility of a cache timing side-channel attack against a T-table–based AES implementation on real hardware. By targeting the L1 data cache and carefully controlling execution order, synchronization, and measurement noise, the attack demonstrates that key-dependent memory accesses can be observed and exploited even on relatively simple in-order cores.

The experimental results confirm that architectural details such as cache geometry, memory alignment, and scheduling behavior play a decisive role in shaping both the strength of the leakage and the practicality of the attack. In particular, the combination of temporal cache sharing, page-aligned lookup tables, and a controlled execution environment enables reliable observation of victim accesses and partial key recovery under the stated threat model.

From a defensive perspective, the findings reinforce the importance of eliminating secret-dependent memory access patterns in cryptographic software. Constant-time implementations remain the most effective countermeasure, as they remove the root cause of cache-based leakage. More generally, architectural and system-level mechanisms such as cache isolation, deterministic memory regions, and careful scheduling policies can substantially reduce the attack surface, albeit often at the cost of performance or flexibility. The results therefore

illustrate the recurring trade-off between efficiency and side-channel resistance in modern systems.

Future work could extend this study in several directions. Possible avenues include attacking the shared L2 cache under weaker scheduling assumptions, improving accuracy on full multi-round AES executions, and evaluating the effectiveness of architectural mitigation mechanisms available on the target platform. Such extensions would provide a broader understanding of how cache-based side channels manifest beyond controlled experimental settings and how they can be mitigated in practice.

References

[1] Daniel J. Bernstein. Cache-timing attacks on aes. 2005.

[2] Microchip Technology Inc. *PolarFire SoC Microprocessor Subsystem Technical Reference Manual*, 2023.

[3] Dag Arne Osvik, Adi Shamir, and Eran Tromer. Cache attacks and countermeasures: The case of aes. *Journal of Cryptographic Engineering*, 2009.